SQLMAP

Open-source tool to detect & exploit SQL Injection (SQLi) vulnerabilities in web apps

Automates SQLi testing by trying multiple payloads & techniques

Cross-platform (Windows, Linux, macOS)

Available as Python scripts: https://github.com/sqlmapproject/sqlmap

Usage

sqlmap -u <URL> [options]

URL: vulnerable web page's link with target parameter (being input values to the server after a ?)

Common Options:

--dbs Enumerate available databases

-D <DB> Select a specific database

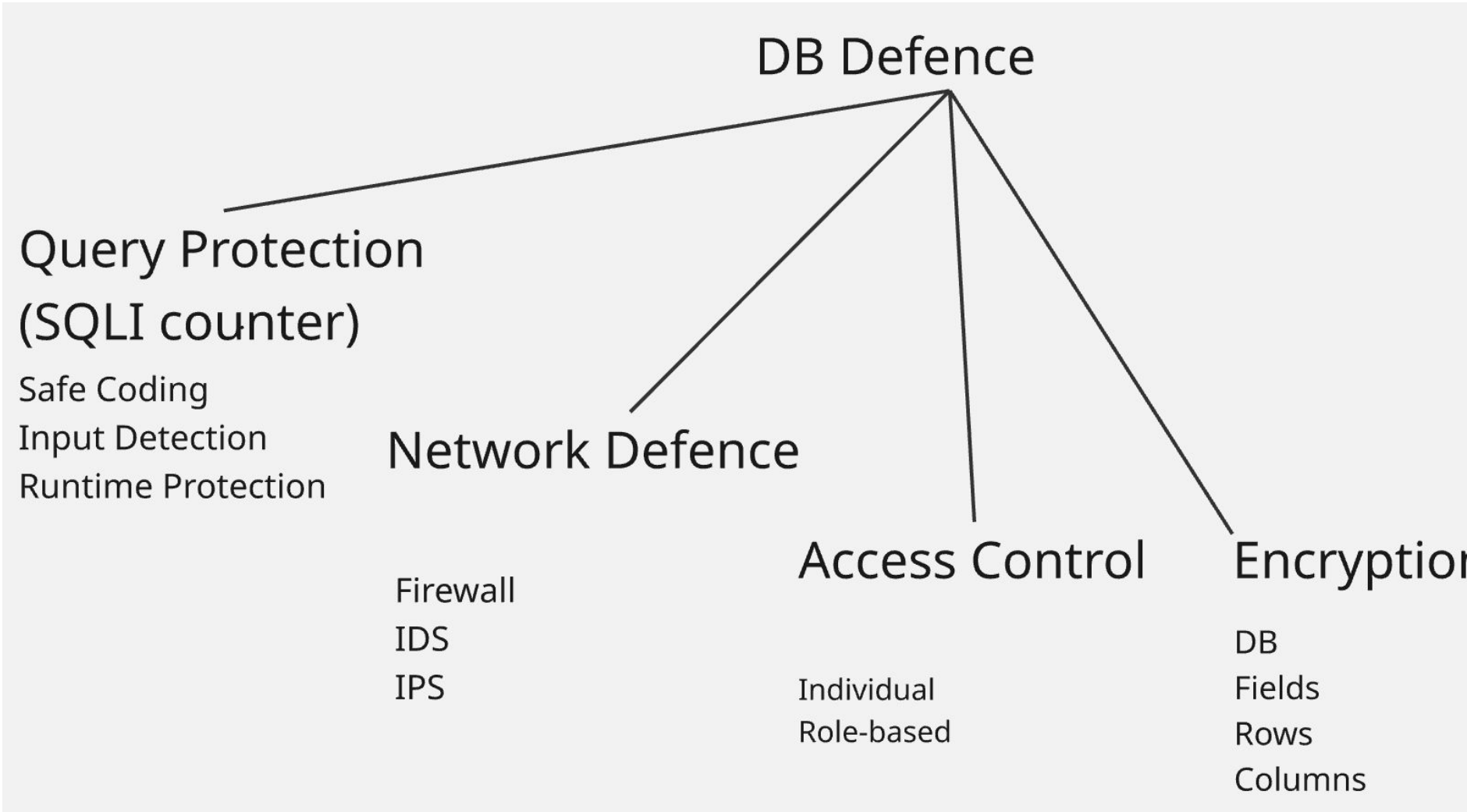
--tables List tables inside chosen DB

-T <TABLE> Select a specific table

--columns List columns of a specific table

-C <COLUMN> Select a specific column

--dump Dump/extract table/column data


DB Defence
Query Protection
(SQLI counter)
Safe Coding
Input Detection
Runtime Protection
Network Defence
Firewall
IDS
IPS
Access Control
Individual
Role-based
Encryption
DB
Fields
Rows
Columns

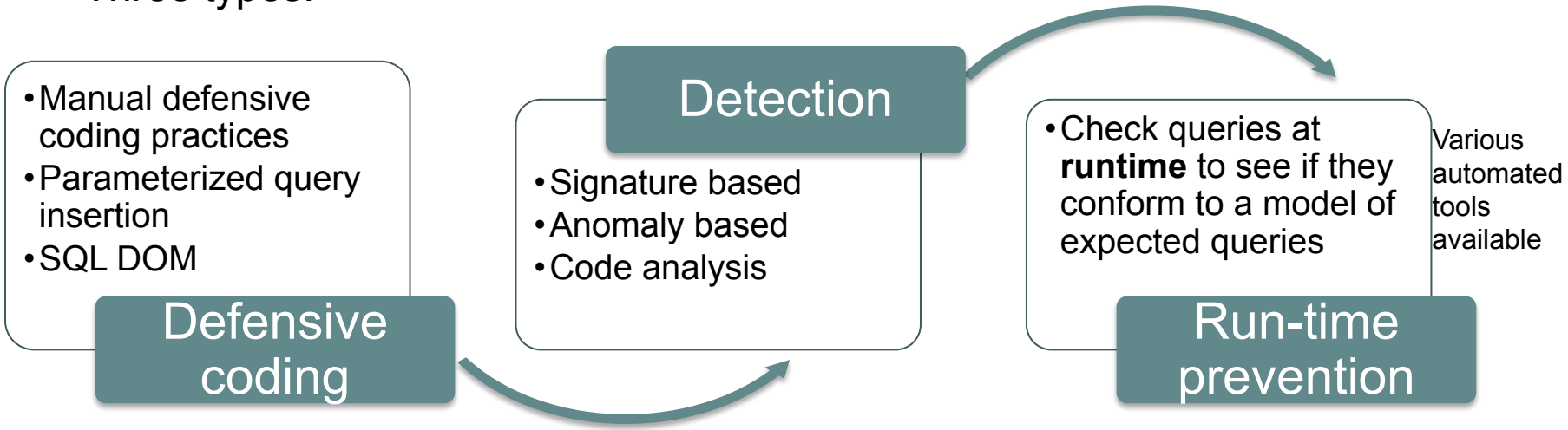
SQLi Countermeasures

Signature based: tries to match specific attack patterns in **input**: must be constantly updated

Anomaly based: defines a normal behavior for **input** and then detects behavior outside normal

Code analysis: looking for exploitable vulnerabilities in the code **before deployment**

Three types:



Manual: input type checking

Parameterized Query: specify the structure of SQL query, taking values as parameters not allowing to modify the query structure

SQL Document Object Model: Set of classes that enables automated data type validation and to construct SQL statements programmatically rather than through direct string manipulation

Parameterized Query

Queries where placeholders are used for user input. No direct string concatenation.

Prevents SQL Injection by treating input as data, not executable SQL

Supported by Python (cursor.execute), Java (PreparedStatement) PHP (PDO::prepare) C# (SqlCommand.Parameters)

Python example

```
conn = sqlite3.connect('example.db')
cursor = conn.cursor()
username = input("Enter username: ") # User input
# Parameterized query
cursor.execute("SELECT * FROM users WHERE username = ?", (username,))
```

SQL DOM

SQL DOM": "SQL Domain Object Model." uses typed query objects
https://dl.acm.org/doi/10.1145/1062455.1062487

Framework handles the query creation: Developer writes query with methods/objects that framework translates to safe SQL.

SQL DOM goes further from Parameterized queries (that prevent injection) with object oriented query building + type safety

Implemented in Frameworks like C#/.NET, SQLAlchemy (Python), jOOQ (Java)

E.g. (C#/.NET)** → `db.Users.Where(u => u.Age > 18)`

Database Encryption

- The database is typically the most valuable information resource for any organization
  - Protected by multiple layers of security
    - Firewalls, authentication, general access control systems, DB access control systems, database encryption
    - **Encryption becomes the last line of defense in database security**
  - Can be applied to the entire **database**, at the **record** level, the **attribute** level, or level of the individual **field**
- **Challenges for encryption:**
  - **Key management:** Authorized users must have access to the decryption key for the data for which they have access
  - **Inflexibility in Search:** When part or all of the database is encrypted it becomes more difficult to perform record searching. **Solution for Search?**

To enable search:

Each individual item in the database is encrypted separately using the same encryption key.

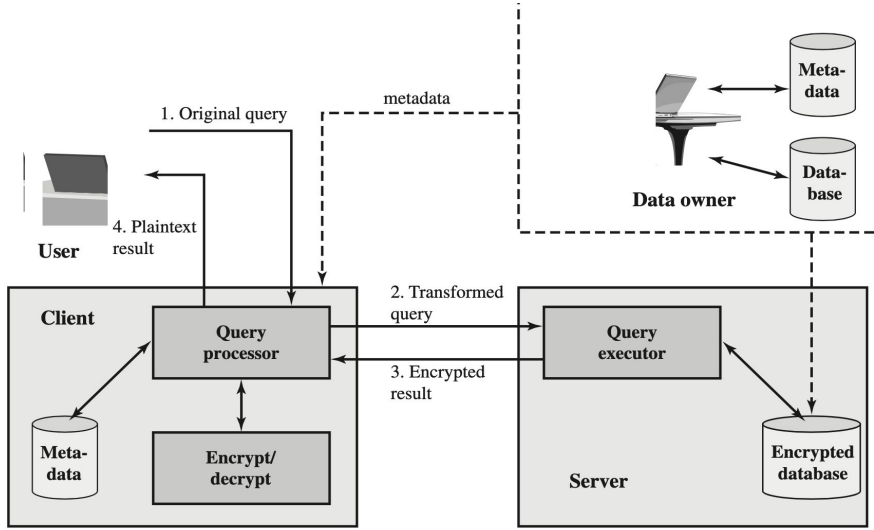
$$\begin{matrix} E(K, x_{11}) & \cdots & E(K, x_{1j}) & \cdots & E(K, x_{1M}) \\ \vdots & & \vdots & & \vdots \\ E(K, x_{i1}) & \cdots & E(K, x_{ij}) & \cdots & E(K, x_{iM}) \\ \vdots & & \vdots & & \vdots \\ E(K, x_{N1}) & \cdots & E(K, x_{Nj}) & \cdots & E(K, x_{NM}) \end{matrix}$$

1. user issues an SQL query with a specific value of the primary key.

2. The query processor at the client encrypts the primary key, modifies the SQL query accordingly, and transmits the query to the server.

3. The server processes the query using the encrypted value of the primary key and returns the appropriate record or records.

4. The query processor decrypts the data and returns the results.



A Database Encryption Scheme

Server can not see the DB

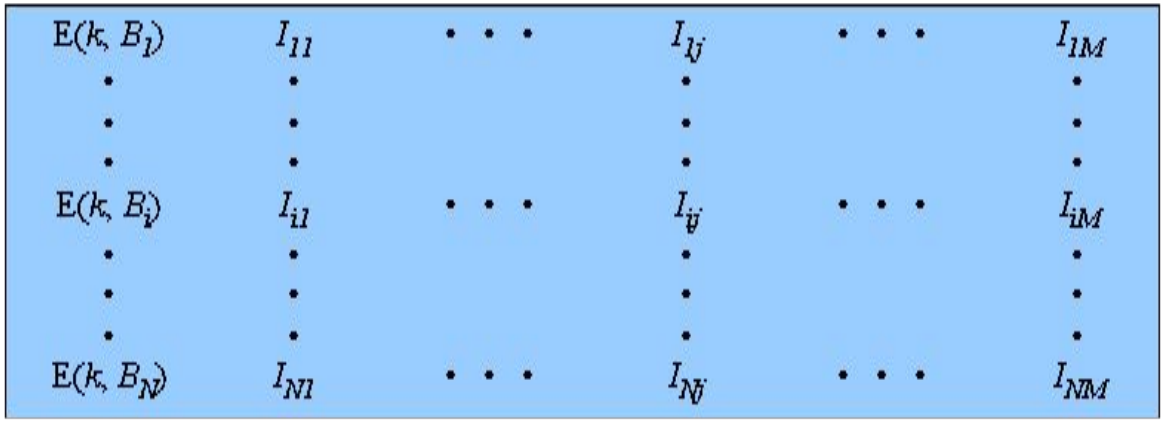
Range search, and sorting not possible

To enable range search and sorting

The entire row is encrypted, concatenating the attributes.

For any attribute, the range of attribute values is divided into a set of non-overlapping partitions that encompass all possible values, and an index value is assigned to each partition.

All the attribute indices for a row are mapped to the row for range/sorted search



$B_i = (x_{i1} \| x_{i2} \| \ldots \| x_{iM})$

Figure 5.10 Encryption Scheme for Database of Figure 5.3

Database Access Control

Database access control system determines:

If the user has access to the entire database or just portions of it

What access rights the user has (create, insert, delete, update, read, write)

Can support a range of administrative policies

Centralized administration

- Small number of privileged users may grant and revoke access rights

Ownership-based administration

- The creator of a table may grant and revoke access rights to the table

Decentralized administration

- The owner of the table may grant and revoke authorization rights to other users, allowing them to grant and revoke access rights to the table

SQL Access Controls

- Two commands for managing access rights:
  - **Grant**
    - Used to grant one or more access rights or can be used to assign a user to a role
  - **Revoke**
    - Revokes the access rights
- Typical access rights are:
  - Select
  - Insert
  - Update
  - Delete
  - References

GRANT { privileges | role }
[ON table]
TO { user | role | PUBLIC }
[IDENTIFIED BY password]
[WITH GRANT OPTION]

Privileges: access rights e.g. select insert etc.
Role: pre-defined group of privileges or users
GRANT OPTION: right to authorize others for same

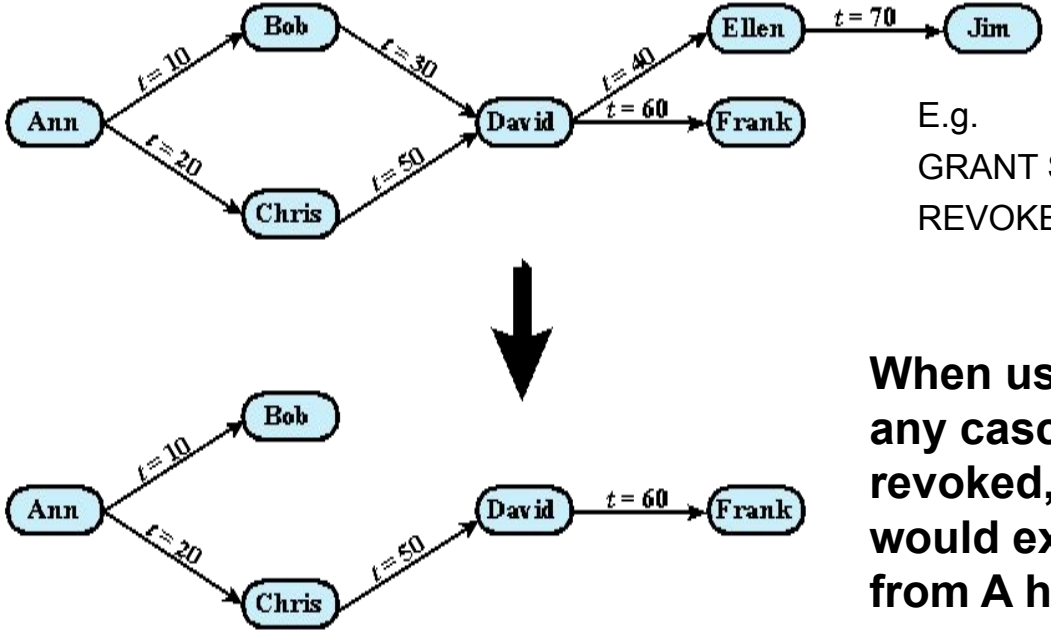


Figure 5.6 Bob Revokes Privilege from David

E.g.

GRANT SELECT ON ANY_TABLE TO David

REVOKE SELECT ON ANY_TABLE FROM David

When user A revokes an access right, any cascaded access right is also revoked, unless that access right would exist even if the original grant from A had never occurred.

If grants are cascaded (using WITH GRANT OPTION) the are revoke is also cascaded from the entire tree for all those provided prior to the redundantly provided existing grant

Role-Based Access Control: (RBAC)

- Role-based access control eases administrative burden and improves security

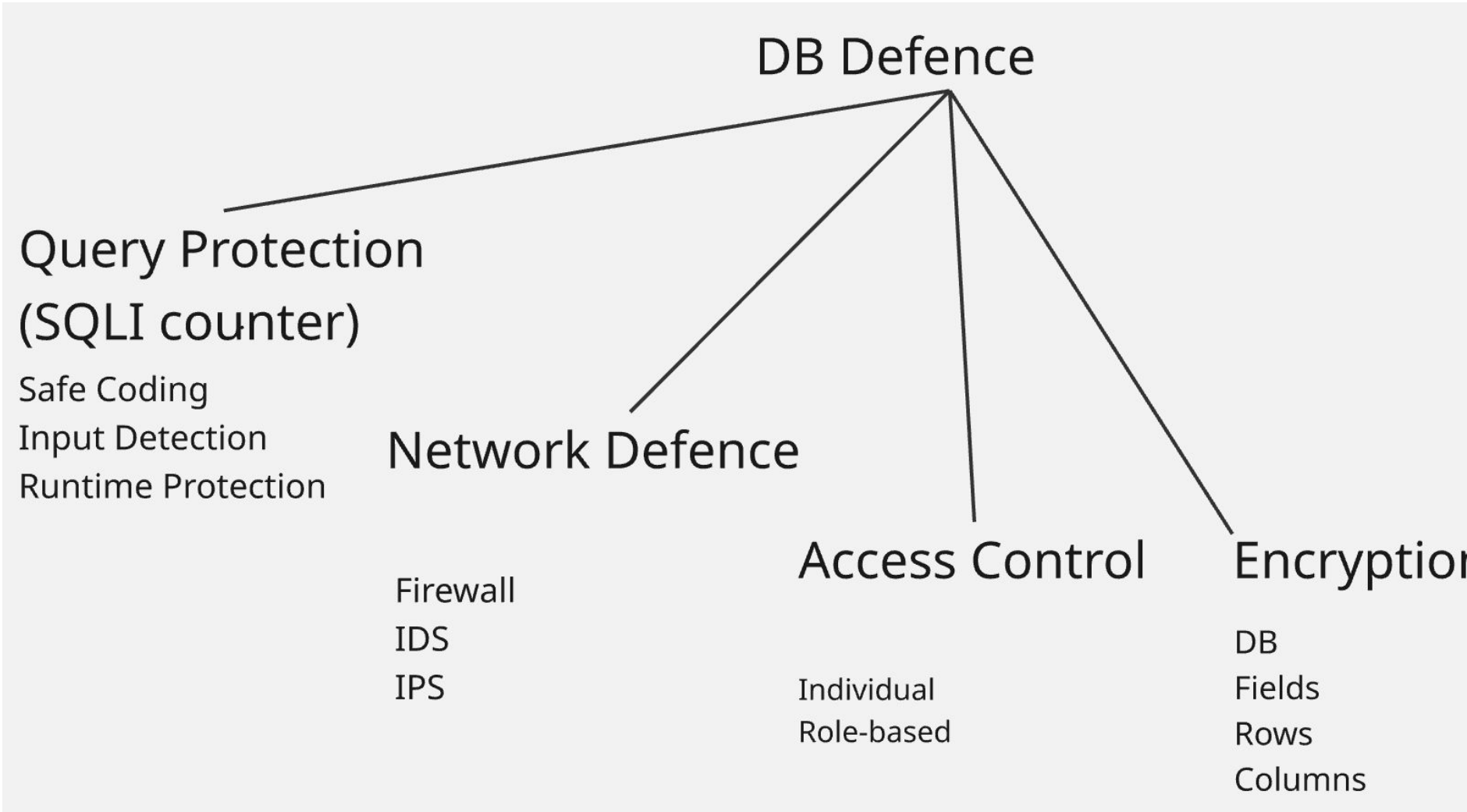
- A database RBAC needs to provide the following capabilities:
  - Create and delete roles
  - Define permissions for a role
  - Assign and cancel assignment of users to roles

- Categories of database users (typical roles):

Application owner	End user	Administrator
• An end user who owns database objects as part of an application	• An end user who operates on database objects via a particular application but does not own any of the database objects	• User who has administrative responsibility for part or all of the database

Example Roles

Role	Permissions
Fixed Server Roles	
sysadmin	Can perform any activity in SQL Server and have complete control over all database functions
serveradmin	Can set server-wide configuration options, shut down the server
setupadmin	Can manage linked servers and startup procedures
securityadmin	Can manage logins and CREATE DATABASE permissions, also read error logs and change passwords
processadmin	Can manage processes running in SQL Server
dbcreator	Can create, alter, and drop databases
diskadmin	Can manage disk files
bulkadmin	Can execute BULK INSERT statements
Fixed Database Roles	
db_owner	Has all permissions in the database
db_accessadmin	Can add or remove user IDs
db_datareader	Can select all data from any user table in the database
db_datawriter	Can modify any data in any user table in the database
db_ddladmin	Can issue all Data Definition Language (DDL) statements
db_securityadmin	Can manage all permissions, object ownerships, roles and role memberships
db_backupoperator	Can issue DBCC, CHECKPOINT, and BACKUP statements
db_denydatareader	Can deny permission to select data in the database
db_denydatawriter	Can deny permission to change data in the database


DB Defence
Query Protection
(SQLI counter)
Safe Coding
Input Detection
Runtime Protection
Network Defence
Firewall
IDS
IPS
Access Control
Individual
Role-based
Encryption
DB
Fields
Rows
Columns